

AMOS: A Memory Operating Layer for Autonomous Data Analysis

Abstract

This document is the product architecture and build guide for AMOS. It defines the theory, system boundaries, data model, runtime protocol, component interfaces, security model, deployment topology, operating model, and phased implementation plan needed to build AMOS from an empty repository to a production service. The intended reader is a startup engineering team that must make the system work, operate it safely, and know when each layer is complete.

AMOS is a model-neutral operating layer between autonomous data-analysis applications and a company's data systems. It converts changing schemas, metrics, snapshots, documents, permissions, prior analyses, reviewer corrections, and execution records into governed analytical memory. For each task, it compiles a bounded working set, grants scoped tool capabilities, validates the versions used, verifies the computation, records claim-level support, and commits the artifact with a replay and invalidation contract. Models propose plans and interpretations; deterministic services retain authority over access, execution, validation, and publication.

The architecture is deliberately incremental. A startup first ships a modular monolith that completes one read-only analytical workflow end to end. It then adds durable feedback, scheduling, multi-tenant governance, connector breadth, and controlled autonomy only after explicit correctness, security, reliability, and customer-value gates pass. The guide defines the complete subsystem map and the normative contracts that detailed designs must implement. Appendices A–F freeze the most dangerous first-release contracts; database DDL, generated schemas, and adapter conformance fixtures remain versioned engineering artifacts rather than claims hidden in prose.

1 Purpose, Audience, and Definition of Complete

1.1 Product Definition

AMOS is the control and memory plane for autonomous data analysis. It is not a warehouse, a dashboard, a general document search engine, or a particular language model. It is the layer that makes analytical work persistent, permissioned, temporally valid, inspectable, and repeatable across models and applications.

The product has three surfaces over one substrate:

1. **Analytical memory:** a typed and versioned representation of the company's metrics, schemas, data states, documents, policies, decisions, prior work, and feedback.
2. **Agent runtime:** a governed execution environment for planning, read-only SQL, deterministic statistics and chart tools, verification, artifact construction, review, scheduling, and recovery. General Python is a later capability behind the same worker interface.
3. **Application platform:** APIs and templates for recurring business reviews, investigations, monitors, forecasts, planning workflows, and domain-specific analytical agents.

1.2 Who Uses This Guide

The primary readers are the founding engineer, data-systems engineer, agent-runtime engineer, product engineer, security owner, and design-partner lead. Sections 2–5 define why the system works. Sections 6–12 define what to build. Sections 13–16 define how to ship, operate, test, and expand it.

1.3 Normative Language and Completeness Standard

MUST: marks a release-blocking invariant. **SHOULD:** marks the default whose exception requires an architecture decision record. **MAY:** marks an optional compatible behavior. **DEFERRED:** identifies a capability intentionally excluded from the first release. **NON-GOAL:** identifies behavior outside the product boundary. Unlabeled explanatory text describes rationale; the appendices contain the executable first-release contract.

The architecture map is complete only when an engineer can answer all of the following without inventing a new subsystem:

- where every governing definition, permission, execution, claim, and artifact is stored;

- how identity and policy are enforced before model-visible context is constructed;
- how a task moves from admission to publication, abort, repair, or human review;
- how source versions are observed and revalidated at commit time;
- how every important claim resolves to computation and governing state;
- how feedback changes future work without silently rewriting history;
- how a source change invalidates dependent conclusions;
- how the service is deployed, monitored, recovered, and tested;
- which capabilities are intentionally deferred and what gate unlocks them.

2 The Product Problem and System Boundary

An analytical task depends on independently changing state. At time t :

$$\mathcal{S}_t = (q_t, D_t, C_t, M_t, X_t, H_{<t}, F_{<t}, P_t), \quad (1)$$

where q_t is the request or trigger, D_t is the data snapshot or stream position, C_t is schema and catalog state, M_t is metric and semantic state, X_t is governed document state, $H_{<t}$ is prior analytical work, $F_{<t}$ is reviewed feedback, and P_t is identity and policy.

These sources do not share one transaction manager. A metric can change after retrieval, a table can migrate during execution, a watermark can advance, and a permission can be revoked before publication. A correct-looking answer can therefore be state-invalid even when its SQL ran successfully. The product problem is to control this cross-source state transition without copying all company data into a prompt or replacing the systems of record.

2.1 Responsibilities

AMOS owns:

- typed metadata, governed summaries, external references, aliases, versions, authority, and supersession;
- identity-aware context construction and task-scoped capabilities;
- plan, tool, verification, claim, artifact, review, replay, and invalidation records;
- a stable runtime contract above replaceable models, indexes, warehouses, and application surfaces.

Source systems continue to own raw data, primary access control, warehouse transactions, catalog records, semantic definitions, document originals, and identity truth. Applications own user experience and domain workflow. Models own no durable authority: model output is a proposal until a deterministic gate accepts it.

2.2 First Product Scope

MUST: The first production slice supports one tenant, one warehouse, one approved metric family, read-only SQL, deterministic built-in statistics, structured chart generation, governed documents, typed report claims, reviewer approval, and replay.

DEFERRED: General Python, free-form claim extraction, arbitrary production writes, unrestricted notebook code, general multi-agent scheduling, unreviewed causal claims, and autonomous external communication. Expansion occurs only through the gates in Section 18.

3 Theory: Analytical Memory as an Operating Layer

3.1 Memory Object

Persistent analytical memory is a set $\mathcal{M}_t$ of typed objects. Each object is:

$$m = (id, k, v, \tau_e, \tau_r, a, p, s, r, h), \quad (2)$$

where k is type, v is content or an external reference, τ_e is the effective-time interval, τ_r is recording time, a is authority, p is permission scope, s is lifecycle status, r is provenance, and h is a content hash. Logical identity and physical version are separate: multiple versions can describe one metric or schema, but only versions valid for the task interval and policy may govern a run.

Core types are data or stream state, schema/catalog state, semantic definition, governed document, prior analysis, feedback, permission policy, execution, artifact, claim, and provenance edge. Raw fact tables remain external; memory stores identifiers, governed summaries, versions, and immutable references.

3.2 Bounded Active Context

The model-visible working set $\mathcal{W}_t \subset \mathcal{M}_t$ is compiled for one task. It is not ordinary top- k retrieval. Selection must satisfy:

$$\text{permitted}(u, p_t, m) \quad \forall m \in \mathcal{W}_t, \quad (3)$$

$$\text{effective}(m, t_q) \wedge \text{active}(m), \quad \forall m \in \mathcal{W}_t, \quad (4)$$

$$\sum_{m \in \mathcal{W}_t} \text{tokens}(m) \leq B, \quad (5)$$

$$\forall r \in R(q) : \exists m \in \mathcal{W}_t \text{ covering role } r. \quad (6)$$

The required-role set $R(q)$ typically includes metric, schema, data state, policy, and supporting evidence. A missing mandatory role produces an explicit non-final outcome rather than a guessed answer. This is the central scale separation: persistent memory may grow to millions of objects while model context remains bounded by B .

3.3 Authority, Time, and Supersession

Recency alone does not determine truth. The default authority order is owner-approved > reviewer-approved > user note > model hypothesis > untrusted external content. Effective time determines when a statement applies; recording time determines when AMOS learned it. Supersession edges explicitly replace one logical version with another. Conflicting owner-approved objects for the same interval block publication until resolved.

3.4 Consistency Classes

Each connector declares the strongest attainable observation class:

- **C0 observed:** the version and observation time are recorded, but the source cannot guarantee re-readability.
- **C1 validated:** the version can be re-read or compared at commit, so changes are detectable.
- **C2 retained:** the exact version is retained or leased long enough for deterministic reconstruction.

Task policy declares minimum classes per required role. Regulated or high-impact artifacts may require C2 data and C1 policy/semantic state; exploratory tasks may accept C0 with a visible warning. **MUST:** A connector may advertise C1 or C2 only after passing the certification behaviors in Specification D; a timestamp label alone is never a version guarantee.

4 The Analytical State Transaction

A-TXN is the system contract connecting memory, tools, validation, publication, and later invalidation. It is not an ACID transaction across external systems. It creates a truthful AMOS commit record after optimistic revalidation of the external versions used.

4.1 Transaction Record

An A-TXN record contains:

$$A = (id, u, q, P^0, V^0, \mathcal{W}, K, E, C, \mathcal{G}, o, t), \quad (7)$$

where P^0 is the admitted policy epoch, V^0 is the source-version vector, K is the scoped capability set, E is execution evidence, C is the claim set, $\mathcal{G}$ is the dependency graph, o is outcome, and t contains lifecycle timestamps.

4.2 Lifecycle

1. **BEGIN:** authenticate identity, bind tenant, classify risk, assign budgets, record policy epoch, and create an idempotent A-TXN ID.
2. **OBSERVE:** ask connectors for source versions, freshness, retention class, and access capabilities.
3. **SELECT:** filter candidates before ranking; reconcile identity, time, authority, and supersession; require role completeness.
4. **PLAN:** produce a typed plan with declared inputs, tool operations, expected outputs, budgets, and review obligations.
5. **EXECUTE:** issue short-lived capabilities to isolated workers; verify each operation before execution; record parameters, hashes, timings, and results.
6. **COMPOSE:** construct typed claims from verified result objects, then render the artifact; prose extraction is not authoritative in the first release.
7. **VERIFY:** validate SQL, schema, metrics, freshness, permissions, claim support, and risk-specific rules.
8. **REVALIDATE:** re-read the policy epoch and every C1 version; confirm C2 retention leases.
9. **COMMIT:** atomically store artifact metadata, claims, dependencies, verification, replay descriptor, and audit event.
10. **DISSEMINATE:** enqueue an idempotent publication request only after evidence commit and object finalization; record external acknowledgment separately.
11. **INVALIDATE:** on later source events, traverse reverse dependencies and update semantic validity, policy visibility, replay availability, review, and supersession dimensions independently.

4.3 Isolation, Concurrency, and Side Effects

MUST: Long model and tool work never holds a database transaction open. Short PostgreSQL transactions use `READ COMMITTED`; every state transition compares the expected `state` and `state_seq`, and every worker lease carries a monotonically increasing fencing token. Terminal states are immutable. A state mutation and its outbox event commit in the same database transaction. Every external side effect uses a stable tenant-scoped idempotency key. Specification A defines the transition, locking, retry, object-promotion, and external-publication protocols.

4.4 Runtime Invariants

The implementation must enforce these invariants:

1. no inaccessible memory enters model-visible context;
2. every selected object is effective, active, non-superseded, and source-qualified;
3. every tool call is allowed by both policy and a task-scoped capability;
4. execution references only declared working-set objects and observed source versions;
5. numeric claims have computation, data-state, schema, and metric support;
6. causal or high-impact recommendations remain review-gated unless an approved design permits automation;
7. commit fails or becomes explicitly non-final when governing C1 state changed;
8. memory writes preserve authority, history, source, and the transaction that produced them;
9. stale workers cannot commit after a newer fencing token is issued;
10. evidence commit, object finalization, and external dissemination have separate persistent states;
11. every final artifact has a declared replay level and complete audit record.

Outcome	Meaning	Required behavior
pass	All required checks succeed.	Commit as final within the declared risk class and replay level.
warning	Work is structurally valid with a disclosed limitation.	Commit as non-final or limited-use; surface the exact warning and affected claims.
repair	A bounded, policy-safe correction is possible.	Record failed attempt, patch the plan or operation, and retry within budget.
needs_review	Evidence exists but judgment or approval is required.	Commit evidence and draft artifact; create a review task; prohibit final publication.
reject	Policy, safety, validity, or support fails.	Store the rejection and diagnostics; do not publish the artifact.
abort	Infrastructure, lease, source, or concurrency failure prevents a truthful decision.	Preserve partial evidence, release capabilities, and allow idempotent retry.

Table 1: Every task ends in an explicit state; silence and best-effort finalization are forbidden.

4.5 Outcome State Machine

5 End-to-End Product Architecture

The startup should implement the first release as a modular monolith with strict internal interfaces. This provides one transactional database, one deployment unit, and one debugging surface while preserving seams for later service extraction.

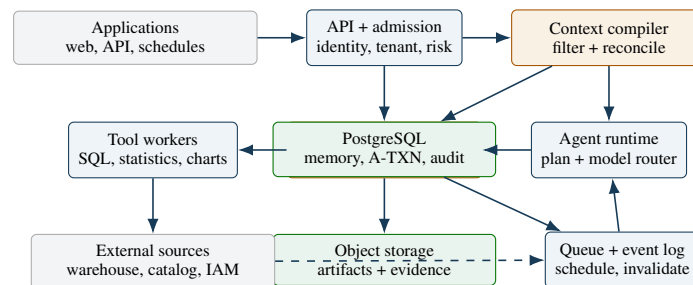


Figure 1: Production logical architecture. Internal boundaries remain explicit even when deployed as one application.

5.1 Component Responsibilities

6 Product Surfaces and Operating Workflows

The control plane is useful only when its state is legible to customers. AMOS therefore ships four product surfaces over the same APIs and permission checks; none receives privileged database access.

6.1 Customer Journey

Onboarding is an auditable promotion workflow. An administrator creates the tenant, connects a read-only source, confirms identity synchronization, and chooses retention and region. The connector discovers schemas and capabilities into a staging namespace. A data owner resolves aliases, defines or imports a metric, assigns authority, and approves a version. A product owner then configures an application template with task schema, required roles, risk class, review rule, schedule, budget, and artifact shape. Production activation is prohibited until connector, permission, metric, task-fixture, and recovery checks pass.

During normal use, a request or schedule creates a transaction and immediately displays its policy scope. The workspace streams durable state changes rather than model tokens as the source of truth. A user can open the exact context manifest,

Component	Owns	Must not own
API and admission	Authentication, tenant binding, request validation, idempotency, risk class, quotas.	Prompt construction, warehouse credentials, or final publication logic.
Connector manager	Source discovery, version observation, freshness, access probes, change events, read handles.	Cross-source reconciliation or company-wide policy decisions.
Memory service	Typed objects, versions, authority, effective time, aliases, supersession, indexes.	Arbitrary raw customer data or model prompts as the system of record.
Context compiler	Required-role derivation, policy-first filtering, ranking, reconciliation, compaction, manifest.	Tool execution or hidden fallback when a required role is missing.
Agent runtime	Typed planning, model routing, budgets, checkpoints, repair loop, artifact draft.	Credentials, unrestricted tools, or authority to bypass verifier outcomes.
Tool workers	Read-only SQL, deterministic statistics, structured charts, result capture; later sandboxed Python behind the same interface.	Long-lived tenant secrets, arbitrary alpha code, or direct publication.
Verifier	SQL safety, schema, metrics, freshness, permissions, claim support, risk and review rules.	General truth or causal certainty beyond declared rules.
Evidence service	Claims, dependency edges, hashes, verification, artifacts, replay descriptors.	Silent mutation of committed evidence.
Scheduler and invalidator	Durable jobs, retries, source-change fan-out, revalidation and review tasks.	Unbounded automatic reruns or publication without current policy.
Review application	Evidence inspection, approve/reject/correct, scoped feedback, audit export.	Direct mutation of historical objects or unscoped high-authority feedback.

Table 2: Ownership boundaries prevent the language model or any single service from becoming an implicit superuser.

Surface	Primary user	Required capabilities
Memory Studio	Data owner and administrator	Connect and test sources; inspect discovered objects and versions; define aliases, metrics, authority, retention, and policies; approve changes; view source health and impact before activation.
Analysis Workspace	Analyst and business operator	Submit or schedule a typed task; see admitted scope, plan, budget, progress, warnings, and terminal state; inspect an artifact down to claim, computation, and source; export approved results.
Review Queue	Reviewer and domain owner	Compare evidence with the draft; approve, reject, or correct at claim, metric, template, or policy scope; assign effective time and authority; see which future tasks and existing claims are affected.
Operations Console	Support, security, and platform operator	Search transactions and audit events; inspect connector, queue, worker, and policy health; retry an idempotent step, revoke a capability, quarantine a connector, revalidate claims, and run recovery procedures.

Table 3: The product exposes governed state and explicit controls instead of presenting an opaque chat transcript.

each execution, each verifier result, and the dependency chain of every material claim. Final status distinguishes *published*, *published with warning*, *awaiting review*, *rejected*, *stale*, and *revoked*; a draft is never visually equivalent to a governed artifact. After review, corrections append new governed objects and show their scope before confirmation. The next applicable transaction selects them by authority and effective time. Source or policy change computes impact, marks dependent claims, and either revalidates, schedules bounded re-analysis, or requests review. The user can always answer: what changed, what AMOS did, what remains valid, and who has authority to resolve the rest.

6.2 Experience Invariants

Every UI action uses the same idempotent API and produces an audit event. Hidden background work has a visible job and owner. Destructive actions are modeled as revocation, supersession, or policy-governed erasure. Restricted object names and failure details are redacted at the API, not merely in the interface. No screen offers a bypass around review, verification, retention, or tenant boundaries.

7 Data Architecture and Persistent Contracts

7.1 Core Relational Model

Use PostgreSQL as the authoritative store. Object storage holds large artifacts, source exports, chart images, and retained execution payloads. A queue transports durable jobs; it does not replace the transaction log.

Table	Primary identity	Required content and rule
tenants	tenant_id	Status, retention policy, encryption key reference, feature gates. Every governed row includes tenant_id.
identities	tenant_id + subject_id	Provider, roles, groups, policy attributes, last synchronized epoch.
source_systems	source_id	Connector type, configuration reference, attainable consistency classes, health. No plaintext secret.
memory_objects	object_id	Type, logical key, source, authority, permissions, effective interval, recorded time, status, content hash, external reference.
memory_versions	logical_key + version	Immutable content or reference, source version, transaction time, supersedes link. Updates append; they do not overwrite.
alias_edges	edge_id	Source-qualified logical identity mapping, authority, effective interval, reviewer state.
policies	policy_id + epoch	Declarative admission, retrieval, tool, publication, retention, and review rules.
policy_decisions	decision_id	Subject, action, resource, attributes, allow/deny, obligations, policy references, epoch, safe explanation.
task_definitions	task_type + version	Required/optional roles, consistency minima, tools, claim types, verifier and publication rules, budgets, artifact schema.
atxn_transactions	atxn_id	Request, identity, risk, budgets, state/state sequence, policy epoch, version vector, timestamps, terminal outcome.
context_manifests	manifest_id	Selected object versions, required-role coverage, omissions, conflicts, token counts, ranking trace.
plans	plan_id	Typed steps, declared inputs/outputs, tools, budgets, dependencies, model/configuration identity.
executions	execution_id	Capability, worker, tool version, parameters hash, input versions, output reference/hash, latency, cost, status.
artifacts	artifact_id	Type, immutable object reference, audience, risk, creator transaction, object-finalization state.
artifact_publications	publication_id	Adapter, idempotency key, lifecycle state, external ID, acknowledgment, attempts, revocation state.
claims	claim_id	Typed payload, rendered span, risk, support plan, review state, semantic validity, policy visibility, replay and supersession state.
dependency_edges	edge_id	Claim-to-query, data, metric, schema, document, execution, feedback, or decision relation.
verification_records	verification_id	Check type/version, inputs, result, warnings/errors, repairability, timestamp.
replay_packages	package_id	Replay level, retained inputs, tool/model versions, parameters, hashes, environment descriptor.
reviews	review_id	Reviewer identity, decision, comment, correction scope, authority, affected claims, timestamp.

Table	Primary identity	Required content and rule
audit_events	event_id	Append-only actor, action, target, request/A-TXN IDs, outcome, policy epoch, timestamp.
jobs	job_id	Schedule/trigger, idempotency key, state, retry policy, lease, fencing token, next run, dead-letter reason.

Table 4: Minimum production schema. JSON payloads may supplement but never replace indexed identity, time, policy, and state columns.

7.2 Memory Object Contract

Every memory object must contain a stable ID, source-qualified logical key, type, human-readable summary, structured payload or external reference, source, authority, effective start/end, recorded timestamp, permission labels, sensitivity, version, status, supersedes/superseded-by links, provenance reference, and content hash. Optional confidence cannot override authority.

Write rules are strict. Connectors may write source-observed objects within their namespace. Reviewed corrections may create reviewer-approved feedback. Only designated owners can create owner-approved definitions. Model-inferred memory starts as a hypothesis and cannot govern a high-risk task until approved. Deletion becomes revocation or tombstoning unless a retention policy requires physical erasure.

7.3 Indexes and Access Paths

MUST: The first release uses relational indexes and PostgreSQL full-text search with mandatory tenant and coarse permission predicates. Required indexes include tenant/type/status, logical key/version, effective interval, permission label, source version, authority, supersession, transaction state, job lease, and reverse dependency target. Object-level authorization runs after candidate retrieval. Restricted candidates cannot affect user-visible counts or ranking explanations. Indexes, embedding caches, lexical caches, and object-store keys are tenant-scoped and policy-epoch-aware. **DEFERRED:** Shared cross-tenant vector indexes. A vector or hybrid backend may later generate candidates only behind the same permission-first interface.

7.4 Retention and Encryption

Encrypt transport and storage. Use per-environment keys initially and tenant-specific keys for enterprise isolation. Store secret references in a secret manager. Define retention independently for prompts, model responses, query results, raw evidence, artifacts, audit events, and C2 snapshots. A replay claim is allowed only when the required data remains retained for its declared interval.

8 Connector Architecture

Each connector implements a small, testable contract:

- `discover(scope)` returns source-qualified entities and capabilities;
- `observe(ref)` returns version, effective time, freshness, sensitivity, access, and consistency class;
- `read(ref, version, capability)` returns governed content or a bounded read handle;
- `validate(ref, observed.version)` confirms whether a C1 observation still holds;
- `subscribe(cursor)` emits version, schema, permission, freshness, or deletion events;
- `health()` reports authentication, rate limits, lag, and degraded capabilities.

Connector output is untrusted until normalized and policy-labeled. Source text never carries executable policy. Each connector receives least-privilege, tenant-scoped credentials and runs behind egress restrictions. Pagination, retries, backoff, rate-limit state, and source cursors are durable.

8.1 Connector Build Order

Build one warehouse adapter first, then identity/policy, semantic definitions, catalog/schema, documents, and event/stream state. Each new adapter must ship with fixtures, contract tests, access-revocation tests, version-change tests, pagination tests, cache-staleness tests, deletion tests, source-outage behavior, and a failure-mode table. C1 certification additionally proves stable version identity, delayed revalidation, overwrite detection, and permission-change observation. C2 additionally proves exact version reads, retention/lease deadlines, expiry behavior, and reconstruction checksums. Do not add a connector solely for breadth; add it when a paid workflow requires one of its roles.

9 Context Compiler

The context compiler turns an approved, versioned task definition into a permission-safe working set and an auditable manifest. **MUST:** Required roles are stored in `task_definitions`; they are never inferred only by a prompt. The record also freezes consistency minima, allowed tools, claim types, verifier rules, publication requirements, budgets, and artifact schema. Specification C provides the contract.

9.1 Compilation Algorithm

1. derive required and optional roles from the task type and risk policy;
2. query candidate indexes with tenant and permission pushdown;
3. remove inaccessible, revoked, expired, and superseded objects before semantic ranking;
4. group by source-qualified logical key and resolve aliases explicitly;
5. reconcile effective time and authority; surface equal-authority conflicts;
6. choose at least one valid object for every mandatory role;
7. fill remaining budget by relevance, authority, freshness, verification state, and diversity;
8. compact only by producing a non-governing derived object with links to every source object;
9. emit a context manifest containing selections, omissions, conflicts, warnings, token cost, and source versions.

9.2 Failure Behavior

Missing role coverage yields `needs_review` or `reject`. Ambiguous approved metrics yield a clarification request. Conflicting owner definitions block. Insufficient token budget never silently removes policy, schema, metric, or data-state roles; it drops optional evidence first. Restricted-object counts and identifiers are not revealed to unauthorized users.

9.3 Compaction Contract

MUST: A compacted summary is non-governing evidence, carries the strictest permission scope of every source, links every source version, and is invalidated when any source changes or is revoked. It may not replace a required metric, schema, policy, or data-state object. At most one summary generation may depend on another summary in the first release. Promotion requires measured token reduction and a regression test for factual and permission leakage. **DEFERRED:** Authority-bearing learned summaries.

10 Agent Runtime and Tool Execution

10.1 Typed Planning

A plan is data, not free-form prose. Each step declares purpose, required object IDs, tool, parameters schema, expected output, dependency steps, timeout, row/data limits, cost budget, retry policy, verification rules, and review obligation. The runtime validates the plan before any capability is issued.

Model routing is policy-driven. A small classifier may assign task type, a coding model may propose SQL, and a reasoning model may plan an investigation. The stored plan records the exact model provider, model ID, configuration, prompt template version, input manifest, latency, token use, and response hash. Models are replaceable processors; persistent memory and safety controls are not provider-specific.

10.2 Capabilities and Workers

The runtime exchanges a plan step for a short-lived, single-purpose capability. A signed capability binds issuer, audience, tenant, A-TXN ID, plan step, identity, tool, allowed source and operations, resource limits, issued/not-before/expiry times, unique token ID, policy epoch, and fencing token. Workers reject expired, mismatched, superseded, or replayed capabilities and record the token ID without storing the bearer token.

MUST: Alpha workers are limited to read-only SQL, deterministic built-in statistics, predefined transformations, and structured chart specifications. SQL workers use source credentials unavailable to the model and enforce approved schemas, statement timeouts, row/byte limits, and query tagging. Every execution writes a hash-addressed result and audit record. **DEFERRED:** General Python requires a separate sandbox safety case covering image provenance, packages, filesystem, network, secrets, resources, determinism, and replay.

10.3 Repair Loop

Repairs are bounded. The verifier may return a machine-readable reason and permitted repair class, such as rename a superseded column or add a required metric predicate. The runtime may attempt at most the policy limit, preserving every failed plan and execution. Permission, policy, or authority failures are never model-repairable.

11 Verification, Claims, and Publication

11.1 Pre-Execution Verification

For SQL, the verifier parses an AST and checks read-only safety, approved relations and columns, metric-required filters, time semantics, groupings, joins, unsafe functions, row limits, and permission scope. It checks schema version, data freshness, snapshot/watermark state, and whether all referenced objects are in the context manifest. Unsupported constructs produce review or rejection; string matching alone is insufficient.

11.2 Post-Execution Verification

After execution, verify result shape, empty or partial outputs, freshness, numeric consistency, denominator rules, uncertainty requirements, chart-data equality, and output sensitivity. **MUST:** The alpha constructs typed claims from verified result objects before rendering prose. Each governed template field declares its claim type and support schema; numeric values never originate from model-written text. **DEFERRED:** Claim extraction from arbitrary prose, notebooks, dashboards, and application payloads is a compatibility feature and cannot be the source of truth.

Claim types include numeric, comparison, trend, forecast, causal, and operational recommendation. Each type has a support plan. Numeric claims require query/result, data state, metric, schema, execution hash, and verification. Causal claims require evidence and remain review-gated unless a pre-approved causal design applies. Recommendations require supporting facts, policy, decision authority, and an explicit reviewer or automation rule.

11.3 Claim Dependency Graph

For claim c , AMOS records edges to computations, data versions, metrics, schemas, documents, feedback, model outputs, verification records, and review decisions:

$$\text{traceable}(c) \not\Rightarrow \text{correct}(c) \not\Rightarrow \text{causally_justified}(c). \quad (8)$$

The graph proves traceability and supports change impact. Verification supports limited consistency. Causal justification remains a separate methodological judgment.

11.4 Publication Linearization Point

Immediately before commit, the runtime revalidates policy epoch and all C1 observations. If any governing state changed, it aborts, repairs, or requests review. The evidence linearization point is the atomic database commit that stores artifact metadata, typed claims, dependencies, verification, replay descriptor, audit event, and object-finalization outbox entry. Large blobs upload first under a temporary key and promote idempotently by content hash.

External dissemination is a separate transaction with explicit forward and revocation paths:

```
draft -> prepared -> evidence_committed -> publication_pending -> published
publication_pending -> publication_failed; revocation_pending -> revoked
```

An outbox worker invokes a publication adapter with a stable `publication_id`; it stores the external identifier and acknowledgment. Lost acknowledgment is recovered by querying or retrying with the same idempotency key, never by creating a second publication. Email, chat, dashboard, ticket, and webhook adapters cannot claim exactly-once behavior unless their target exposes compatible idempotency.

12 Feedback, Learning, and Invalidation

Human feedback is a governed state transition. A correction records author, role, authority, scope, effective interval, target artifact/claims, supporting decision, applicability conditions, and provenance. It never silently edits the old artifact. The next applicable context compilation can select the correction according to time, scope, and authority.

Source events trigger reverse dependency traversal. A changed metric, schema, document, permission, or data state identifies affected claims. **MUST:** Validity is multidimensional: `publication_validity`, `semantic_validity`, `policy_visibility`, `replay_availability`, `review_state`, and `supersession_state` change independently. Thus a claim can remain historically valid at publication time while becoming semantically stale, hidden from a newly unauthorized viewer, or unreplayable after retention expiry. Fan-out is bounded by tenant quotas, risk priority, deduplication keys, and cooldown windows. External notifications occur only after a new evidence commit and current policy decision.

12.1 Replay Levels

Level	Promise	Required retention
0	Disposable exploration	Audit event only; no regeneration claim.
1	Evidence reconstruction	Object versions, queries, documents, results, and review records.
2	Artifact regeneration	Level 1 plus structured artifact inputs, templates, and deterministic render configuration.
3	Computational replay	Level 2 plus C2 data, code, parameters, tool environment, and result hashes.
4	Audit package	Level 3 plus policy, identity, approvals, signed manifests, and exportable chain of custody.

Table 5: Replay language is precise; model trajectory replay is recorded when possible but is not guaranteed.

13 API and Event Contracts

The first public API is versioned under `/v1`. All mutating requests require an idempotency key. Every response includes request ID, A-TXN ID where applicable, status, warnings, and a stable error code.

Events use an envelope containing event ID, type, schema version, tenant, source, subject, occurred/observed times, correlation and A-TXN IDs, payload hash, and trace context. Consumers are idempotent. Required event families are:

- source and policy: `source.version.changed`, `policy.epoch.changed`, `permission.revoked`;
- task and evidence: `task.state.changed`, `artifact.evidence_committed`, `claim.validity_changed`;

Endpoint	Purpose	Contract
POST /tasks	Admit and run or queue analysis	Request/trigger, audience, risk, budgets, required freshness; returns transaction and job IDs.
GET /tasks/{id}	Observe lifecycle	State, current step, warnings, budgets consumed, review obligation; no hidden restricted details.
POST /memory/search	Inspect governed memory	Permission-first typed results and manifest; privileged admin modes are separate.
POST /memory	Create governed memory	Authority and source constrained by caller role; append-only version semantics.
POST /verify/sql	Preflight proposed SQL	Structured pass/warning/reject, rule IDs, repairability, referenced versions.
GET /artifacts/{id}	Retrieve artifact metadata	Publication/validity state, audience, evidence links, replay level.
POST /artifacts/{id}/publications	Request dissemination	Adapter, destination reference, idempotency key; returns publication ID and durable state.
GET /publications/{id}	Inspect dissemination	Lifecycle, external ID, acknowledgment, attempts, last error, revocation state.
GET /claims/{id}	Inspect a claim	Exact span, type, support edges, verification, review, current validity.
POST /reviews	Approve, reject, or correct	Reviewer identity, scoped decision, affected claims, feedback object created.
POST /replay/{id}	Reconstruct or replay	Allowed level, retained inputs, status, differences, new audit record.
POST /artifacts/{id}/revalidate	Check current validity	Traverses dependencies and returns per-claim changes.

Table 6: Minimum API surface. Administrative configuration and connector secrets use separate privileged interfaces.

- dissemination: `artifact.publication_requested`, `artifact.published`, `artifact.publication_failed`;
- workflow: `review.completed`, `job.dead.lettered`.

14 Security and Multi-Tenancy

14.1 Trust Boundaries

Retrieved content, model output, user-provided text, and external tool output are untrusted. Identity tokens, policy decisions, capabilities, source credentials, verification rules, commit logic, and audit events remain outside model text. A successful prompt injection may influence a proposed plan; it must not grant a capability, reveal filtered data, write high-authority memory, or publish restricted output.

Threat	Preventive control	Detection/recovery
Cross-tenant leakage	Tenant key on every row, database row-level security, tenant-scoped indexes/keys/caches.	Canary records, access-log analytics, immediate key and token revocation.
Indirect prompt injection	Evidence/policy separation, capability checks, sandboxed tools, output validation.	Malicious-content flags, plan diffs, red-team corpus, quarantine.
Permission-revocation race	Policy epoch and permission revalidation at commit.	Abort commit, invalidate affected claims, audit event.
Memory poisoning	Authority tiers, source namespaces, review gates, immutable versions.	Conflict alerts, provenance inspection, rollback by supersession.
Credential exposure	Secret manager, worker-side credential exchange, no secrets in prompts/logs.	Automated scanning, short TTL, rotation and incident runbook.
Unsafe SQL or code	AST checks, read-only roles, isolated workers, network/resource limits.	Kill switch, query tags, execution audit, dead-letter evidence.
Provenance leakage	Permissioned graph traversal and redacted support views.	Link-access tests, export review, access anomaly alerts.
Denial of service	Admission quotas, budgets, concurrency limits, fan-out controls, circuit breakers.	Queue depth/latency alerts, tenant isolation, graceful degradation.

Table 7: Security is enforced by external controls, not by asking the model to follow policy.

14.2 Policy Decision Contract

MUST: Policy is declarative and versioned behind one engine-neutral interface:

evaluate(subject, action, resource, task, environment, epoch) → decision + obligations + policy_refs.

The persisted result includes allow/deny, row and column constraints, redactions, review and retention obligations, applied policy versions, input-attribute hash, and a caller-safe explanation. Policies may depend on classified result metadata but not arbitrary model prose. Owners author policy drafts; a separate authorized reviewer activates them after simulation against stored decision fixtures. One epoch identifies an immutable set of policy versions. Runtime caches key on tenant, subject attributes, action, resource version, and epoch; epoch activation evicts the prior namespace. The interface permits Rego, Cedar, or a typed internal evaluator, but product code cannot embed bypass policy in prompts or route handlers.

14.3 Tenant Isolation

MUST: Tenant identity appears in every primary key and every foreign-key path through composite keys; no object ID is globally addressable without tenant binding. PostgreSQL row-level security is forced for runtime tables, and runtime roles cannot use `BYPASSRLS` or own those tables. Migration and runtime roles are separate. Application repositories repeat tenant predicates; property tests cover cross-tenant joins. Cache keys, indexes, object prefixes, encryption context, queues, capabilities, jobs, and audit queries are tenant-scoped. Canary rows and automated database-policy checks continuously detect bypass. Offer dedicated databases, worker pools, and customer-managed keys when required. Every background job restores tenant context explicitly; missing tenant context is a hard failure.

14.4 Privacy and Governance

Minimize retained raw rows. Support field classification, masking, purpose limitation, regional storage, retention, legal hold, export, and erasure. Erasure must update both primary records and derived artifacts while retaining only the minimum audit proof permitted by policy. High-risk model providers receive only the bounded, redacted context allowed by tenant configuration.

15 Deployment and Scalability Architecture

15.1 Initial Production Topology

Deploy the modular monolith as stateless API/runtime replicas behind a load balancer. Use managed PostgreSQL with point-in-time recovery, object storage with versioning, a managed queue, isolated worker pools, secret manager, identity provider, metrics/logs/traces backend, and a private network path to customer sources. Separate development, staging, and production accounts. Infrastructure is declarative and reviewed.

Run interactive and scheduled workloads in separate queues. Partition workers by tool type and risk. Use A-TXN IDs and idempotency keys across retries. Database transactions own state transitions; queue acknowledgements occur only after durable state is written. Long operations renew leases and checkpoints.

15.2 When to Extract Services

Do not distribute the system prematurely. Extract a component only when one of these conditions is measured: tool workers require a stronger isolation boundary; connector traffic has independent scaling; context indexes exceed primary-database capacity; invalidation fan-out interferes with interactive latency; enterprise deployment requires separate data planes; or team ownership demands an independent release cycle. Preserve the same typed contracts when extracting.

15.3 Scaling Strategy

Scale persistent memory and model context independently. Keep structured filters in the database, move large blobs to object storage, and introduce a dedicated lexical/vector candidate index behind the memory interface when needed. Shard by tenant before sharding within a tenant. Maintain reverse dependency indexes for invalidation. Cache only version-addressed, permission-scoped results; policy-epoch changes evict them.

Capacity planning tracks active tenants, memory objects, version rate, dependency edges, scheduled tasks, tool concurrency, result bytes, model tokens, p50/p95/p99 control-path latency, queue age, and invalidation fan-out. Load tests include one-million-object tenants, policy churn, source delay, and noisy neighbors.

16 Operations, Reliability, and Support

16.1 Service Objectives

Initial paid-pilot objectives are 99.5% API availability, 99% successful scheduled runs excluding declared source outages, p95 admission/context compilation below two seconds at the supported scale, zero confirmed cross-tenant or permission leaks, and 100% of final artifacts with complete claim/evidence manifests. Objectives tighten only after error budgets and workload shape are understood.

16.2 Observability

Every request carries request, A-TXN, tenant, job, plan, execution, artifact, publication, and trace IDs. Metrics cover task outcomes, verifier results by rule, context role coverage, retrieval leaks, repair counts, review backlog, capability denials, query cost, model cost, queue age, source health, invalidation lag, publication acknowledgment, replay success, and storage growth. Logs are structured, redacted, immutable according to policy, and linked to traces. Customer-visible status distinguishes AMOS failure from source or provider failure.

16.3 Runbooks

Ship runbooks for source authentication failure, permission leak suspicion, policy epoch mismatch, stuck transaction, worker compromise, runaway query, queue backlog, database failover, object-store mismatch, invalidation storm, replay mismatch, model-provider outage, and data erasure. Each runbook specifies trigger, containment, customer impact, evidence preservation, recovery, owner, and post-incident tests.

Backups are restored in staging on a schedule. Target RPO is five minutes and RTO is one hour for the control plane during paid pilot, with a longer declared window for reconstructible artifacts. Disaster-recovery drills verify database, object, queue, key, and connector configuration recovery together.

17 Testing and Quality Gates

Testing mirrors the architecture rather than only happy-path answers.

17.1 Definition of Done for a Feature

A feature is done only when its typed contract, migration, permissions, audit events, metrics, failure modes, idempotency behavior, retention behavior, tests, operator runbook, and user-facing status are implemented. Model prompt changes alone do not constitute a product feature.

18 Step-by-Step Build Plan

The team advances by exit gates, not calendar optimism.

Phase	Product state	Build	Exit gate
0	Architecture foundation	Freeze Specifications A–F, generated schemas, domain types, migrations, A-TXN state machine, audit log, tenant/identity skeleton, CI, local fixtures, one-command environment.	Contract examples validate; all invariants have executable tests; a failed step is inspectable and idempotently retryable.

Phase	Product state	Build	Exit gate
1	Governed memory	Memory/version schema, authority/time/supersession, PostgreSQL search, permission policy, seed tooling, admin inspection.	Permission, time, authority, and supersession property tests pass; no destructive update path.
2	One connector and context	Certified C1 warehouse adapter, versioned task definition, required roles, context compiler, manifest, permission-first search, ambiguity handling.	Frozen task fixtures always include required roles; zero restricted or superseded retrievals; connector certification passes.
3	Governed answer	Typed planner, SQL/statistics/chart workers, capability claims, AST verifier, structured claims, result hashes, report renderer, task API.	At least 90% safe-and-correct completion on frozen pilot tasks; every numeric claim resolves to evidence.
4	Persistent analyst	Review UI/API, scoped feedback, multidimensional claim validity, replay levels 1–2, schedules, bounded invalidation.	Four weekly shadow cycles complete; corrections affect the next applicable run; validity dimensions change correctly.
5	Paid pilot	SSO/RBAC, defense-in-depth tenancy, object finalization, idempotent publication adapter, durable queues, observability, backups, runbooks, quotas.	Three design partners; zero critical security findings; lost-acknowledgment and recovery drills pass; onboarding under ten business days.
6	Platform API	Stable SDK, connector kit, application templates, model routing, admin controls, retention/erasure, exportable audit packages.	Two independent applications use the same memory/runtime without bespoke core changes.
7	Scale and enterprise	Hybrid indexes, worker/service extraction where measured, dedicated tenancy options, regional deployment, customer-managed keys.	SLOs pass at target tenant/edge/concurrency scale; noisy-neighbor and failover tests pass.
8	Controlled autonomy	Policy-scoped auto-publication, dependency-driven re-analysis, portfolio scheduling, advanced statistics, rollback controls.	Each autonomous task class has a separate safety case, customer approval, monitoring, kill switch, and rollback drill.

Table 9: Implementation sequence from empty repository to controlled autonomy.

18.1 Reference Repository Shape

Preserve architectural boundaries in code before extracting network services. A practical modular-monolith layout is:

apps/api/	admission, public API, event streaming
apps/web/	Memory Studio, workspace, review, operations
amos/domain/	IDs, value objects, states, errors, interfaces
amos/memory/	objects, versions, authority, time, indexes
amos/connectors/	connector SDK and source adapters
amos/context/	role schemas, selection, reconciliation, manifests
amos/runtime/	planner, controller, budgets, checkpoints, repair
amos/workers/	isolated SQL, statistics, chart workers
amos/verification/	policy, SQL, result, claim, publication checks
amos/evidence/	claims, dependencies, artifacts, replay packages
amos/scheduler/	jobs, leases, retries, invalidation, revalidation
amos/policy/	identity, authorization, risk and retention rules

Layer	Required tests	Release gate
Unit/property	Time intervals, authority ordering, supersession, policy evaluation, state transitions, CAS conflicts, fencing, hash stability.	Deterministic and exhaustive boundary cases; mutation/property tests for core invariants.
Connector contract	Versions, revocation, pagination, retry, cursor recovery, access denial, rate limit, source deletion.	Same suite passes every connector; no adapter-specific bypass.
Context compiler	Role completeness, permission-first filtering, conflicts, ambiguity, budget pressure, compaction lineage.	Zero restricted/superseded selections; mandatory roles never silently dropped.
Verifier	Valid/invalid SQL corpus, metrics, schema drift, freshness, unsafe functions, claim support.	Published precision/recall thresholds and zero acceptance of seeded critical violations.
End to end	Request, schedule, review, feedback, replay, invalidation, abort/retry, artifact export.	Every terminal outcome and recovery path exercised with persisted evidence.
Security	Cross-tenant, prompt injection, poisoning, capability replay, secret leakage, provenance inference, DoS.	Zero critical leaks; red-team findings have regression tests.
Reliability	Worker kill, stale lease, database failover, duplicate events, object promotion, lost publication acknowledgment, source timeout, model outage.	No duplicate publication; fenced, idempotent recovery preserves audit trail.
Performance	Memory/edge growth, concurrency, churn, fan-out, noisy neighbor, queue saturation.	SLOs hold at declared paid-pilot limits with recorded capacity margin.
Human review	Evidence usability, defect detection, correction scope, audit time.	Reviewers can locate support and submit bounded corrections without engineer help.

Table 8: A release is not qualified because a demo answer looks correct; each control boundary must pass its own oracle.

infra/migrations/ specs/contracts/ specs/conformance/ deploy/ tests/	forward database migrations and data backfills generated JSON Schema/OpenAPI and fixtures connector, worker, policy, publication test kits local, staging, production and recovery definitions unit, contract, integration, security, E2E, load
--	---

Dependencies point inward to `amos/domain`; domain code imports no framework, database driver, model SDK, or connector. Packages call another package through a typed interface and may not write its tables. Transactional state change and an outbox event commit together before asynchronous work begins. External adapters implement ports owned by the calling domain. Architecture tests enforce allowed imports, table ownership, tenant-scoped repositories, and the absence of credentials from model-facing packages.

18.2 First Vertical Slice

Use a payment-health or subscription-health review as the first commercial workflow. It is recurring, economically meaningful, bounded, and reviewable. Limit the alpha to one metric family, one warehouse, three query shapes, one chart, one report template, and one reviewer role. Shadow the existing analyst process before replacing any deliverable.

The vertical slice is complete when a user request or schedule produces: admitted transaction; versioned context manifest; verified SQL executions; result and chart hashes; structured claims; review-aware report; claim dependencies; replay package; audit trail; and explicit terminal outcome. A fluent report without these records is not an AMOS product slice.

18.3 Team and Ownership

A five-person initial team is sufficient: founding architect/product lead; memory/connectors engineer; runtime/tools engineer; product/review engineer; and design-partner/data lead. Assign one named owner for security and operations even if the role is fractional. Every architectural component in Table 3 has a code owner and operational owner before paid pilot.

19 Current Prototype to Production

The repository already demonstrates useful seams: Python domain models, SQLite-backed memory, permission filtering, temporal/authority reconciliation, a planner/controller, DuckDB execution, SQL verification, claim provenance, replay, FastAPI routes, scenarios, and an 85-test regression suite. This proves the vertical control loop can run. It is not yet the complete production architecture.

Area	Prototype	Production completion work
State store	SQLite tables and local artifacts.	PostgreSQL migrations, row-level security, HA, PITR, object storage, retention, tenant keys.
Transactions	Controller sequence and audit records.	Explicit A-TXN state machine, CAS transitions, fencing, policy/source revalidation, outbox, atomic evidence commit.
Connectors	Synthetic DuckDB and fixture-shaped adapters.	Versioned connector SDK, real warehouse/IAM/semantic/document adapters, change subscriptions.
Runtime	One payment workflow and templated plan.	Versioned task definitions, typed plans, model router, durable checkpoints, signed capabilities, SQL/statistics/chart workers, repair budgets.
Verification	Read-only SQL, schema, metric, freshness, permissions, provenance.	Supported SQL fragment spec, result/statistical checks, rule versioning, claim-type policies, false-reject monitoring.
Provenance	Claim records, support links, replay hashes.	Structured claim schemas, general dependency graph, review decisions, validity dimensions, reverse invalidation, signed audit export.
API	FastAPI task/memory/claim/artifact/replay routes.	Versioning, idempotency, publication lifecycle, pagination, quotas, stable error taxonomy, event schemas, SDK.
Operations	Local execution and evaluation harness.	Queue, observability, SLOs, on-call, backups, DR, metering, customer status, incident runbooks.
Security	Permission filters and read-only tools.	SSO/RBAC, tenant isolation, secret manager, worker sandbox, egress controls, red-team program.

Table 10: The production plan extends the existing prototype without confusing local feasibility with a deployable product.

20 Architecture Decisions and Non-Goals

Key decisions are explicit:

- build a modular monolith first, with internal service contracts and one authoritative transaction database;
- keep raw data in source systems and move bounded results or retained snapshots only under policy;
- enforce permissions before retrieval ranking and again at execution and commit;
- treat models as untrusted, replaceable processors with no ambient credentials;
- use optimistic cross-source validation rather than claim nonexistent global ACID guarantees;
- attach provenance at claim granularity and define replay levels precisely;
- construct typed claims before prose and persist validity dimensions separately;
- separate evidence commit, object finalization, and external dissemination;
- gate causal and high-impact conclusions on human or approved methodological review;
- add autonomy per task class, never as a platform-wide switch.

DEFERRED: General Python, free-form claim extraction, vector retrieval, and automatic external publication. **NON-GOAL:** Universal enterprise knowledge management, unrestricted natural-language access to all data, arbitrary production writes, perfect model reproducibility, automatic causal discovery, and replacing warehouses, semantic layers, catalogs, lineage systems, or BI tools.

21 Harsh Qualification Checklist

Before claiming that AMOS is ready for a paid pilot, answer yes to every item:

1. Can an operator reconstruct why every selected memory object entered the context and why every omitted mandatory role caused a visible outcome?
2. Can permission revocation between retrieval and publication prevent commit without relying on the model?
3. Can every numeric statement in a final artifact resolve to query, result hash, data version, metric, schema, and verification?
4. Can a reviewer correction affect the next applicable task while the original artifact remains immutable?
5. Can a metric or schema change identify and reclassify every dependent claim?
6. Can duplicate requests, events, or retries occur without duplicate publication?
7. Can a worker be killed or compromised without exposing long-lived credentials or corrupting transaction history?
8. Can the team restore database, objects, queue state, keys, and connector configuration within the declared RTO?
9. Can a tenant export its audit package and enforce retention or erasure without another tenant being observable?
10. Can a lost publication acknowledgment be retried without creating a duplicate external artifact?
11. Do composite tenant keys, forced RLS, repositories, caches, object paths, and capabilities all reject cross-tenant access?
12. Can model provider, retrieval backend, or worker implementation change without changing the memory and A-TXN contracts?
13. Do load, security, connector, failure-injection, and human-review tests pass at the declared supported scale?
14. Does the product save measurable analyst/reviewer time while maintaining zero confirmed critical permission leaks?

A no answer is an architectural gap, not documentation debt. The corresponding phase remains incomplete until the gap has an owner, contract, test, metric, and runbook.

22 Conclusion

AMOS becomes a product when persistent company knowledge, model reasoning, tool execution, verification, human judgment, and artifacts are joined by one enforceable operating contract. Typed analytical memory provides durable state. The bounded context compiler supplies only valid and permitted working memory. A-TXN makes admission, version observation, capabilities, validation, commit, dissemination, replay, and invalidation explicit. Claim dependencies make conclusions inspectable and change-aware. External enforcement keeps models useful without making them trusted authorities.

The implementation path is deliberately practical: build one governed read-only workflow in a modular monolith; make every outcome and dependency observable; add durable review, scheduling, tenancy, operations, and connector breadth; then introduce platform APIs and autonomy only when phase-specific gates pass. If the team follows the contracts and qualification checklist in this guide, it can build AMOS end to end without leaving the hardest correctness, security, and operational decisions implicit.

A Specification A: A-TXN, Concurrency, and Publication

This specification freezes the internal state machine and the boundary between evidence commit, object finalization, and external dissemination. Generated enums and transition tests must match this section.

A.1 Persistent State Machine

Current state	Allowed next state	Required guard and durable effect
admitted	observing, rejected	Identity, tenant, task definition, risk, budgets, policy epoch, and request idempotency key are fixed.
observing	selecting, needs_review, aborted	Required connector observations and attainable consistency classes are stored.
selecting	planning, needs_review, rejected	Manifest covers every required role; omissions and conflicts are explicit.
planning	executing, needs_review, rejected	Typed plan validates against the task definition and budget.
executing	composing, repairing, aborted	Every execution uses the current lease fence and a valid capability; results are hash-addressed.
repairing	executing, needs_review, rejected	Failure is in an allowed repair class and the attempt budget remains.
composing	verifying, aborted	Typed claims and artifact inputs are stored before prose render.
verifying	revalidating, repairing, needs_review, rejected	Versioned verifier results cover every required rule and claim.
revalidating	evidence_committed, repairing, needs_review, rejected, aborted	Policy epoch, C1 observations, C2 leases, task version, and publication obligations still hold.
evidence_committed	object_finalizing, publication_pending, needs_review	Claims, dependencies, replay descriptor, audit event, and outbox records commit atomically.
object_finalizing	publication_pending, object_failed	Temporary objects promote by content hash; mismatch never mutates committed evidence.
publication_pending	published, publication_failed	Adapter receives stable publication ID and stores external acknowledgment or classified failure.
published	revocation_pending	Only policy, review, or invalidation may request revocation; evidence remains immutable.
revocation_pending	revoked, publication_failed	Adapter records target acknowledgment; retries retain the same publication ID.
needs_review	verifying, revalidating, rejected	A current authorized review supplies the exact missing obligation.
rejected, aborted, revoked	none	Terminal outcome and evidence are immutable; a retry creates or resumes only through an explicit retry record.

Table 11: Allowed A-TXN and publication transitions. A generated transition matrix is the implementation source of truth.

A.2 Isolation, Compare-and-Swap, and Fencing

MUST: Each database interaction is a short PostgreSQL `READ COMMITTED` transaction. Transition code performs compare-and-swap on tenant, A-TXN ID, current state, and `state_seq`; success increments the sequence. A zero-row update means a concurrency conflict, not success.

```

UPDATE atxn_transactions
SET state = :next_state,
    state_seq = state_seq + 1,
    updated_at = clock_timestamp()
WHERE tenant_id = :tenant_id
AND atxn_id = :atxn_id
AND state = :expected_state
AND state_seq = :expected_seq
AND terminal = false;

```

Job acquisition uses `SELECT ... FOR UPDATE SKIP LOCKED`, increments `fencing_token`, and commits before work begins. Every checkpoint, execution, object promotion, outbox completion, and publication attempt compares that fence. A renewed lease never reuses an old fence. Terminal records reject updates at both repository and database layers. Review, invalidation, and publication commands lock the affected artifact row and compare its validity sequence before appending a new state.

MUST: Every mutation and its outbox event share one database commit. Event consumers deduplicate by tenant and event ID. Side effects use the stable key `tenant/atxn/step/effect`; publication uses `tenant/publication_id`. Duplicate requests return the original resource when the request hash matches and return `IDEMPOTENCY_CONFLICT` when it differs.

A.3 Normal Task and Commit-Time Revocation

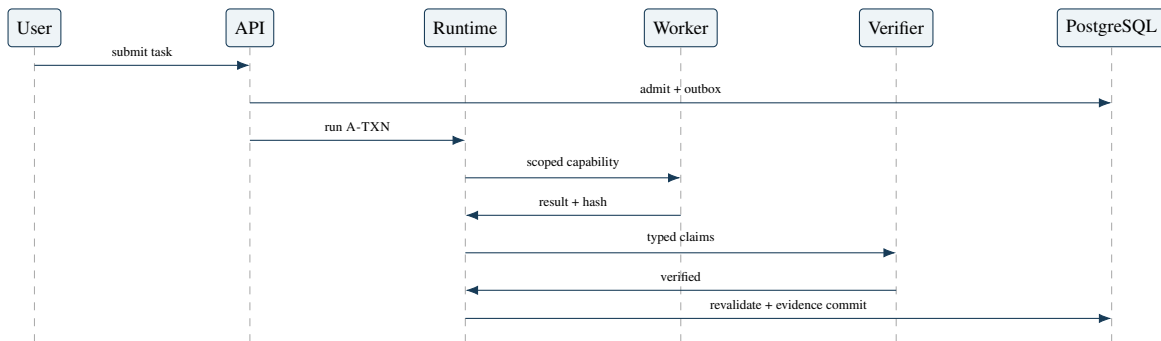


Figure 2: Normal task lifecycle. Durable state, not streamed model text, advances the product.

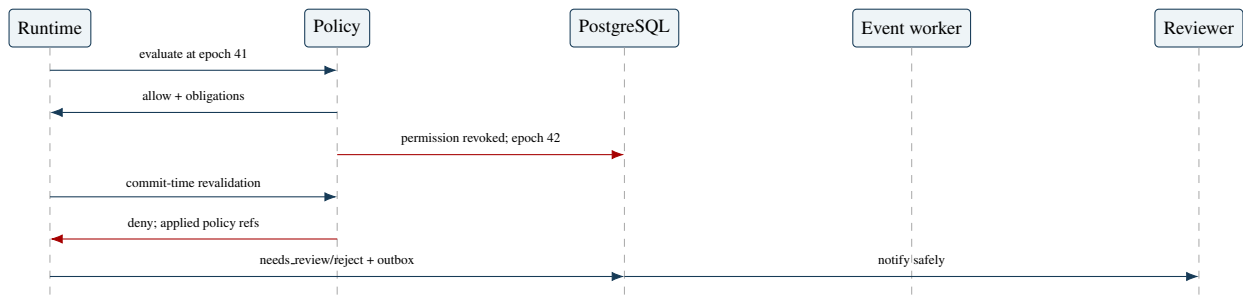


Figure 3: Commit-time policy revocation prevents publication without relying on the model.

A.4 External Publication Protocol

The publication adapter interface is `prepare`, `publish`, `status`, and `revoke`. `prepare` validates destination, audience, current policy, and payload hash without a side effect. `publish` accepts a stable publication ID. `status` reconciles lost acknowledgment. `revoke` is idempotent and records whether the target can truly retract or only append a correction. Permanent failures move to dead letter with operator ownership; retryable failures use capped exponential backoff and preserve the same ID.

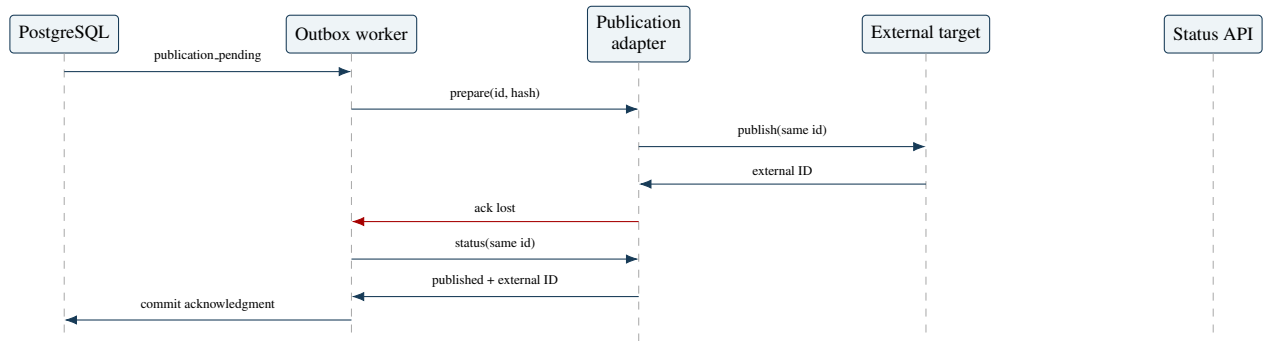


Figure 4: External publication recovers a lost acknowledgment without duplicating the target artifact.

B Specification B: Persistent Model and Database Invariants

B.1 Keys, Constraints, and Roles

Invariant	Implementation rule
Tenant binding	Every primary key and foreign key includes <code>tenant_id</code> . Lookups require tenant plus local ID. Composite foreign keys prevent cross-tenant references.
Forced RLS	Runtime tables enable and force row-level security. Runtime roles neither own tables nor hold <code>BYPASSRLS</code> ; migration roles are separate and unavailable to the application.
Immutability	Memory versions, executions, evidence commits, claims, dependencies, reviews, and audit events are append-only. Correction uses supersession or a new validity record.
State checks	Database checks constrain enum values, nonnegative budgets, effective intervals, terminal flags, and publication transition pairs. Repository CAS supplies transition legality.
Version identity	Unique constraints cover tenant, source, logical key, source version, and content hash where appropriate. Equal version with unequal content is a connector incident.
Idempotency	Unique tenant-scoped keys exist for requests, events, executions, object promotions, jobs, reviews, and publication attempts.
Ownership	Each package owns its tables; other packages use typed ports. Direct cross-package writes fail architecture tests and database grants.
Audit	Sensitive changes append actor, action, target, decision, policy epoch, request, A-TXN, correlation, and time. Audit payloads are redacted but tamper-evident.

Table 12: Database invariants are repeated at schema, database-role, repository, and test layers.

Required indexes begin with tenant and cover active logical version, effective interval, authority, permission label, source version, A-TXN state sequence, job readiness/lease, publication state, and reverse dependency target. A migration that changes a state or contract ships with forward DDL, bounded backfill, compatibility window, verification query, rollback or roll-forward plan, and observed capacity cost. Destructive column removal waits until all running application versions stop reading it.

B.2 Example A-TXN Record

```
{
  "tenant_id": "ten_01",
  "atxn_id": "atxn_01J2",
  "task_definition": {"type": "payment_health_review", "version": 3},
  "request_hash": "sha256:...",
  "subject_id": "usr_42",
  "risk": "material_internal",
  "state": "revalidating",
  "state_seq": 12,
  "policy_epoch": 41,
  "source_versions": {
    "metric:payment_failure_rate": "v3",
    "schema:payments": "etag:82ab",
    "snapshot:warehouse": "snap_2026_07_12T1900Z"
  },
  "manifest_id": "ctx_91",
  "plan_id": "plan_17",
  "fencing_token": 8,
  "outcome": null,
  "created_at": "2026-07-12T19:02:11Z",
  "updated_at": "2026-07-12T19:03:27Z"
}
```

B.3 Object Finalization and Retention

Temporary objects use a tenant prefix and a random upload ID. The evidence commit stores expected hash, size, media type, retention class, and final content-addressed key. Promotion copies or renames idempotently, verifies hash and encryption context, then appends finalization state. A mismatch quarantines the temporary object and leaves evidence inspectable but non-publishable. Garbage collection deletes unreferenced temporary objects only after the maximum retry window.

Retention evaluates each evidence class independently. Erasure creates a tombstone, removes primary and derived bytes, invalidates dependent summaries, and retains only the minimum policy-permitted proof. C2 leases record expiry and renewal; replay availability changes when a lease expires even if historical publication validity remains true.

C Specification C: Task Definitions and Context Manifests

C.1 Versioned Task Definition

```
task_type: payment_health_review
version: 3
status: approved
risk_class: material_internal
required_roles:
  - metric_definition
  - active_schema
  - data_snapshot
  - user_policy
  - review_policy
optional_roles:
  - prior_incident
  - deployment_event
  - reviewer_feedback
minimum_consistency:
  metric_definition: C1
  active_schema: C1
  data_snapshot: C2
  user_policy: C1
allowed_tools:
  - sql.readonly.v1
  - stats.rate_comparison.v1
  - chart.timeseries.v1
claim_types:
  - metric_value
  - metric_comparison
  - operational_recommendation
verifier_profile: payment_health.v2
publication_policy: human_review_required
budgets: {max_steps: 8, max_repairs: 2, max_rows: 50000}
artifact_schema: payment_health_report.v3
```

Task definitions are immutable after approval. A new version declares compatibility, migration behavior for schedules, fixture corpus, owner, reviewer, effective interval, and superseded version. Admission binds one version for the entire A-TXN. Dynamic policy may add obligations or deny execution but cannot silently remove required roles or verifier rules.

C.2 Example Context Manifest

```
{
  "manifest_id": "ctx_91",
  "tenant_id": "ten_01",
  "atxn_id": "atxn_01J2",
  "task_definition": "payment_health_review:v3",
  "policy_epoch": 41,
  "required_role_coverage": {
    "metric_definition": ["mem_metric_pfr_v3"],
    "active_schema": ["mem_schema_payments_82ab"],
    "data_snapshot": ["mem_snapshot_1900"],
    "user_policy": ["mem_policy_access_41"],
    "review_policy": ["mem_policy_review_9"]
  },
  "optional_selected": ["mem_incident_217"],
  "omissions": [{"role": "deployment_event", "reason": "no_authorized_candidate"}],
  "conflicts": [],
  "token_count": 6840,
  "source_versions": ["metric:v3", "schema:82ab", "snapshot:1900"],
  "ranking_trace_ref": "obj://ten_01/traces/ctx_91"
}
```

C.3 Selection and Compaction Rules

Candidate SQL always applies tenant, active status, effective-time overlap, coarse permission scope, and non-superseded filters before text ranking. Object authorization then evaluates the exact subject and policy epoch. The response reveals neither pre-authorization count nor restricted candidate identity. A cache entry is keyed by tenant, task version, subject-attribute hash, policy epoch, query hash, and source-version vector.

Required roles select valid governing objects before optional relevance ranking. Equal-authority conflicts block. Budget pressure removes optional evidence first. A compacted object inherits the strictest source permission and shortest retention, is non-governing, links all source versions, is invalidated on any source change, and cannot satisfy metric, schema, policy, or data-state roles.

D Specification D: Connector SDK and Certification

D.1 Typed Connector Interface

```
discover(scope, cursor?) -> Page<EntityRef>
observe(ref) -> Observation {
  source_version, observed_at, effective_interval,
  freshness, sensitivity, access_descriptor,
  consistency_class, retention_deadline?
}
read(ref, source_version, capability) -> BoundedHandle
validate(ref, source_version) -> Validation {same, current_version, reason}
subscribe(cursor) -> Page<SourceEvent>
health() -> Health {status, lag, rate_limit, degraded_capabilities}
```

Version identity must represent content or source transaction state strongly enough for the advertised class. Connector configuration declares whether updates overwrite identities, how deletions and permission changes surface, cache behavior, clock assumptions, pagination consistency, cursor expiry, and source outage semantics. Equal source version with different normalized content is a hard incident.

D.2 Certification Matrix

Behavior	C0	C1	C2	Oracle
Stable observation identity	required	required	required	Repeated observation without change returns equal version.
Delayed revalidation	—	required	required	Change after delay returns unequal version or unavailable.
Overwrite and deletion	observed	required	required	Fixture overwrite/deletion cannot validate old state.
Permission change	observed	required	required	Revocation becomes observable within declared bound.
Pagination/cursor recovery	required	required	required	No silent duplicate, omission, or cross-scope item.
Cache staleness bound	declared	enforced	enforced	Bypass/expiry test meets declared maximum.
Exact version read	—	optional	required	Old version reads exact bytes and checksum.
Retention lease	—	—	required	Deadline, renewal, expiry, and missing snapshot are deterministic.
Source unavailable	required	required	required	Returns classified unavailable, never a guessed validation.

Table 13: A connector earns its consistency class through conformance tests, not self-description.

D.3 Example Source-Change Event

```
{
  "event_id": "evt_src_882",
  "schema_version": 1,
  "type": "source.version.changed",
  "tenant_id": "ten_01",
  "source_id": "warehouse_primary",
  "subject": "schema:payments",
  "previous_version": "etag:82ab",
  "current_version": "etag:91cc",
  "change_kind": "schema",
  "occurred_at": "2026-07-12T20:00:00Z",
  "observed_at": "2026-07-12T20:00:09Z",
  "cursor": "cur_10921",
  "deduplication_key": "warehouse_primary/schema:payments/etag:91cc"
}
```

E Specification E: Workers, Capabilities, and Verification

E.1 Capability Claims

```
{
  "iss": "amos-runtime",
  "aud": "sql-worker",
  "tenant_id": "ten_01",
  "atxn_id": "atxn_01J2",
  "plan_id": "plan_17",
  "step_id": "step_03",
  "subject_id": "usr_42",
  "tool": "sql.readonly.v1",
  "source_id": "warehouse_primary",
  "operations": ["query"],
  "relations": ["analytics.payments"],
  "limits": {"seconds": 30, "rows": 50000, "bytes": 5000000},
  "policy_epoch": 41,
  "fencing_token": 8,
  "jti": "cap_7701",
  "nbf": 1783908200,
  "exp": 1783908260
}
```

Capabilities are signed by a dedicated issuer, audience-bound, short-lived, single-use for side-effecting operations, and exchanged only over authenticated worker channels. The worker fetches source credentials from the secret broker after capability validation; the model and runtime plan never receive them. Audit stores token ID and claims hash, not the bearer token.

E.2 Typed Plan and Verifier Result

```
{
  "plan_id": "plan_17",
  "task_definition": "payment_health_review:v3",
  "manifest_id": "ctx_91",
  "steps": [{
    "step_id": "step_03",
    "purpose": "compute current and baseline failure rates",
    "tool": "sql.readonly.v1",
    "input_object_ids": ["mem_metric_pfr_v3", "mem_schema_payments_82ab"],
    "parameter_schema": "payment_rate_query.v2",
    "expected_output_schema": "rate_comparison.v1",
    "timeout_seconds": 30,
    "limits": {"rows": 50000, "bytes": 5000000},
    "retry": {"max_attempts": 2, "repair_classes": ["COLUMN_SUPERSEDED"]},
    "verifier_profile": "payment_health.v2"
  }]
}
```

```
{
  "verification_id": "ver_551",
  "execution_id": "exec_123",
  "verifier_profile": "payment_health.v2",
  "profile_version": 2,
  "outcome": "pass",
  "checks": [
    {"rule_id": "SQL_READ_ONLY", "outcome": "pass"},
    {"rule_id": "METRIC_FILTERS", "outcome": "pass"},
    {"rule_id": "SCHEMA_VERSION", "outcome": "pass"},
    {"rule_id": "RESULT_DENOMINATOR", "outcome": "pass"}
  ],
  "warnings": [],
  "permitted_repair": null,
  "input_hash": "sha256:...",
  "created_at": "2026-07-12T19:03:11Z"
}
```

E.3 Worker Protocol and Deferred Python

The worker validates capability signature, audience, time, tenant, source, operation, policy epoch, and fence; reserves the execution idempotency key; obtains a short-lived source credential; validates the operation; executes within limits; writes result bytes under a temporary tenant key; commits output hash and classified status; destroys credentials; and acknowledges only after durable state. Cancellation and lease loss stop execution and prohibit commit.

DEFERRED: A general Python worker is not an alpha component. Its later safety case must define immutable runtime images, allowlisted packages and hashes, no ambient network or filesystem, secret absence, syscall and resource isolation, data-egress controls, deterministic seed/time behavior, result capture, vulnerability response, and replay claims.

E.4 Verifier Repair and Worker Crash Sequences

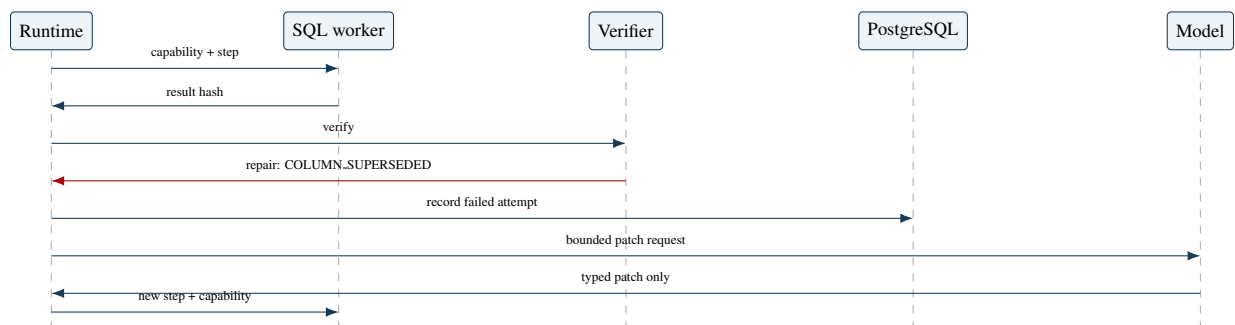


Figure 5: Verifier repair is bounded, typed, and preserves the failed attempt.

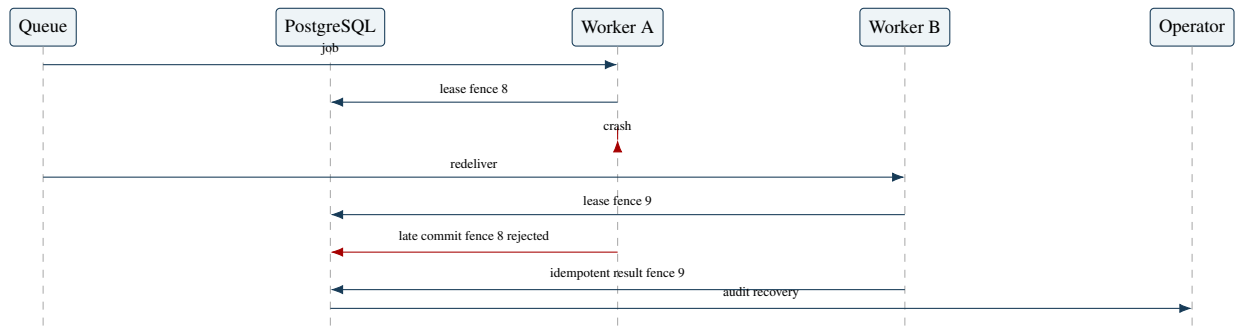


Figure 6: A worker crash cannot let a stale lease corrupt transaction history.

F Specification F: Claims, Review, Invalidation, and Replay

F.1 Structured Claim and Dependency Edge

```

{
  "tenant_id": "ten_01",
  "claim_id": "clm_501",
  "artifact_id": "art_700",
  "claim_type": "metric_comparison",
  "payload": {
    "metric_id": "payment_failure_rate:v3",
    "current_value": 0.074,
    "baseline_value": 0.022,
    "unit": "ratio",
    "window": {"start": "2026-07-05", "end": "2026-07-12"}
  },
  "support_execution_ids": ["exec_123"],
  "verification_ids": ["ver_551"],
  "publication_validity": "valid_at_publication",
  "semantic_validity": "current",
  "policy_visibility": "allowed",
  "replay_availability": "level_2",
  "review_state": "verified",
  "supersession_state": "active"
}
  
```

Dependency edge

```
{
  "edge_id": "edge_991",
  "tenant_id": "ten_01",
  "from": {"type": "claim", "id": "clm_501"},
  "relation": "computed_by",
  "to": {"type": "execution", "id": "exec_123"},
  "source_version": "snapshot:1900",
  "created_by_atxn": "atxn_01J2",
  "content_hash": "sha256:..."
}
```

F.2 Review Correction, Replay Package, and API Error

Review correction

```
{
  "review_id": "rev_881",
  "tenant_id": "ten_01",
  "artifact_id": "art_700",
  "claim_ids": ["clm_501"],
  "reviewer_id": "usr_finance_owner",
  "decision": "correct",
  "correction": {
    "type": "metric_filter",
    "value": "exclude_internal_accounts",
    "scope": "task:payment_health_review",
    "effective_from": "2026-07-13T00:00:00Z",
    "authority": "owner_approved"
  },
  "original_artifact_mutated": false
}
```

Replay package

```
{
  "package_id": "rpl_440",
  "tenant_id": "ten_01",
  "artifact_id": "art_700",
  "replay_level": 2,
  "manifest_id": "ctx_91",
  "plan_id": "plan_17",
  "execution_ids": ["exec_123"],
  "template": "payment_health_report:v3",
  "render_config_hash": "sha256:...",
  "retained_until": "2027-07-12T00:00:00Z",
  "expected_artifact_hash": "sha256:..."
}
```

API error response

```
{
  "request_id": "req_909",
  "atxn_id": "atxn_01J2",
  "error": {
    "code": "CONTEXT_REQUIRED_ROLE_MISSING",
    "message": "A required context role is unavailable.",
    "retryable": false,
    "safe_details": {"role": "active_schema"},
    "review_required": true
  }
}
```

F.3 Validity Transition Rules

Dimension	Representative states	Transition cause
Publication validity	draft, valid_at_publication, publication_failed, revoked	Evidence and review state at publication; external adapter acknowledgment; later revocation.
Semantic validity	current, pending_revalidation, stale, invalid	Metric/schema/data/document change and verifier outcome.
Policy visibility	allowed, redacted, denied	Current subject, audience, policy epoch, retention, or legal hold.
Replay availability	level_0–level_4, degraded, expired	Retained evidence, C2 lease, tool environment, template, and policy package.
Review state	unreviewed, needs_review, approved, corrected, rejected	Authorized review append; correction never overwrites prior evidence.
Supersession state	active, superseded, tombstoned	New artifact/claim version, owner action, or erasure policy.

Table 14: Validity dimensions remain independent so historical truth is not confused with current usability.

F.4 Review and Source-Change Sequences

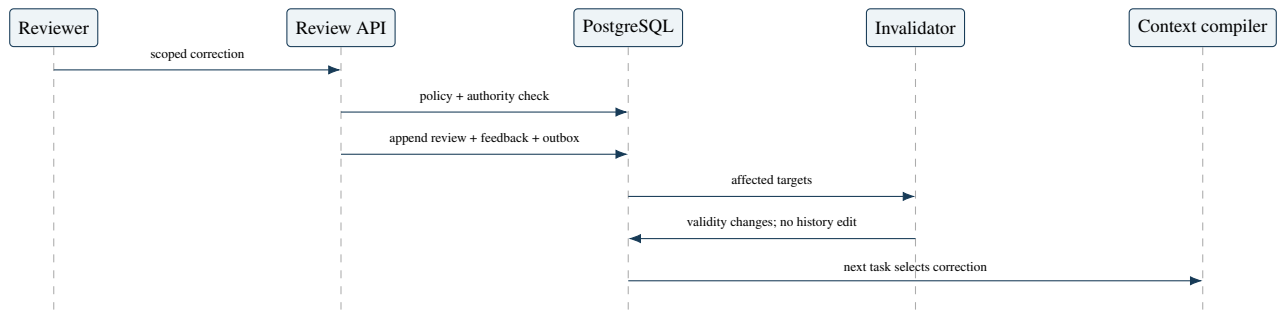


Figure 7: Human review creates governed feedback while preserving the original artifact.

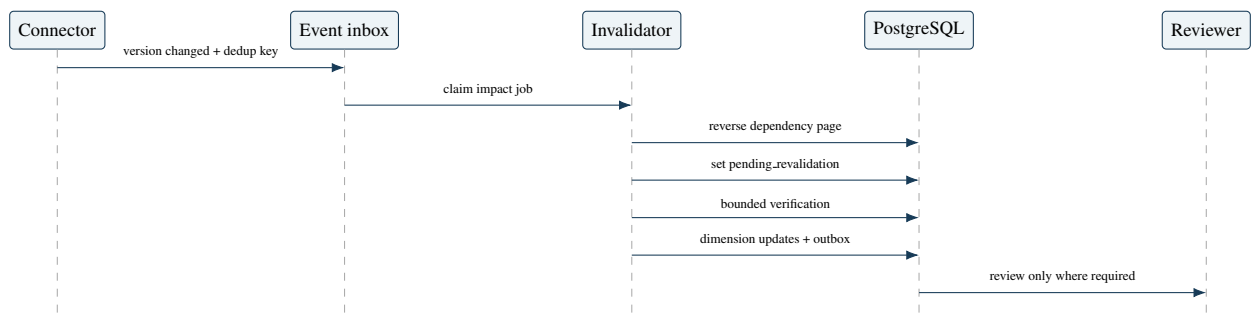


Figure 8: Source-change invalidation is deduplicated, paged, bounded, and dimension-specific.

F.5 Replay Sequence

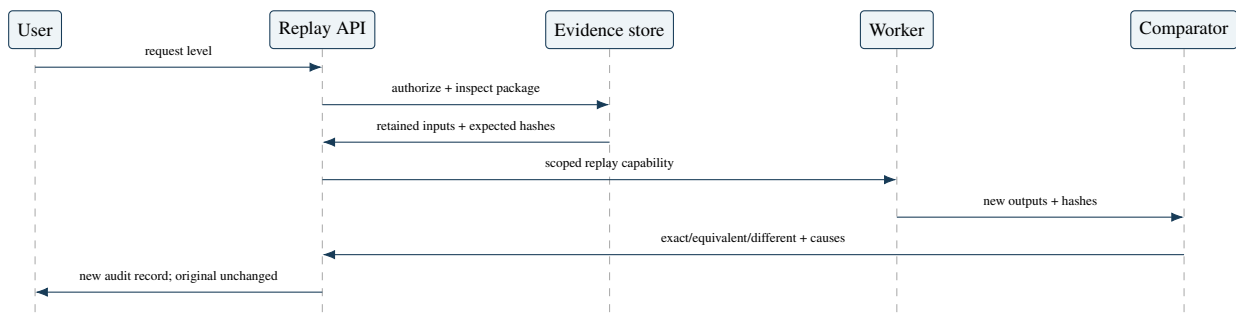


Figure 9: Replay creates new evidence and a comparison; it never rewrites the original run.